EASWARI ENGINEERING COLLEGE (AUTONOMOUS)

DEPARTMENT OF INFORMATION TECHNOLOGY

B.TECH DEGREE UNIT TEST-II, JULY 2026 — III Year IT / VI Semester

231ICSE903T — NATURAL LANGUAGE PROCESSING

Consolidated Answer Key — Merged from Set A and Set B | Date: 22.07.2026

Mark Distribution Summary

Part	Structure	Marks per Question	Total Marks
Part A	5 short-answer questions (5 from Set A + 5 from Set B)	2 marks each	10
Part B	1 compulsory questions (1 from each set)	8 marks each	8
Part C	2 questions, each with an OR alternative — both alternatives answered here	7 marks each	7
		Grand Total (both papers combined)	25

PART – A (10 × 2 = 20 Marks)

Short answers to all Part A questions from both question papers, presented in the order they appear, each within approximately four lines as required.

1. What are the two main components of NLP? *[2]*

The two main components of NLP are Natural Language Understanding (NLU) and Natural Language Generation (NLG). NLU deals with analysing and interpreting human language to derive meaning, structure and intent from text or speech. NLG deals with converting structured data or internal machine representations back into fluent, human-readable natural language. Together they allow a system to both comprehend input and produce coherent output.

2. Explain the importance of text normalization in NLP. *[2]*

Text normalization converts raw, inconsistent text into a standard, uniform format — for example, converting to lowercase, expanding contractions, removing punctuation and correcting spelling variants. It reduces vocabulary size by treating different surface forms of the same word as one, which improves the accuracy of tasks such as search, parsing and machine learning models. It also removes noise like special characters, extra whitespace and inconsistent number/date formats. Overall, it acts as a cleaning step that makes downstream NLP processing more reliable and efficient.

3. What is a grammar? *[2]*

A grammar is a formal set of rules that defines the syntactic structure of legal sentences in a language, describing how words and phrases combine correctly. It typically consists of terminals (actual words), non-terminals (syntactic categories like NP, VP) and production rules relating them. Formalisms such as Context-Free Grammar (CFG) are widely used in NLP to parse sentences and validate structure. Grammars form the theoretical basis for syntactic analysis and parser design.

4. Why is NLP difficult? *[2]*

NLP is difficult mainly because natural language is inherently ambiguous at the lexical, syntactic, semantic and pragmatic levels, so the same words or sentence can have multiple valid interpretations. Language also relies heavily on context, world knowledge and common sense that machines do not naturally possess. It contains irregularities such as idioms, sarcasm, slang and constantly evolving vocabulary that do not follow fixed rules. In addition, variability across dialects, languages, and informal text (e.g., social media) makes building a single general-purpose system very challenging.

5. Define Tokenization in NLP. *[2]*

Tokenization is the process of breaking a continuous stream of text into smaller meaningful units called tokens, such as words, sub-words or punctuation marks. It is typically the very first step in the text pre-processing pipeline before any further linguistic analysis is done. For example, the sentence “NLP is fun” is tokenized into the tokens [“NLP”, “is”, “fun”]. Tokenization enables subsequent operations such as stop-word removal, stemming, POS tagging and frequency counting.

6. Define NLP. *[2]*

Natural Language Processing (NLP) is a subfield of Artificial Intelligence and computational linguistics that enables computers to understand, interpret, manipulate and generate human language. It combines linguistic rules with statistical and machine-learning techniques to process both written text and spoken language. NLP underlies applications such as machine translation, chatbots, sentiment analysis and search engines. In essence, it bridges the gap between human communication and machine-processable data.

7. Differentiate NLU and NLG. *[2]*

Natural Language Understanding (NLU) focuses on the comprehension side — interpreting unstructured human language input and converting it into a structured, machine-usable representation, e.g., extracting intent or entities from a sentence. Natural Language Generation (NLG), in contrast, focuses on the production side — converting structured data or internal representations into fluent, natural human-readable language, e.g., generating a report or a chatbot reply. NLU can therefore be thought of as "input/comprehension" and NLG as "output/production." Most conversational systems use both together to understand a query and then respond naturally.

8. List any four applications of NLP. *[2]*

Four common applications of NLP are: (1) Machine Translation, e.g., Google Translate converting text between languages; (2) Sentiment Analysis, used to gauge opinions in product reviews or social media; (3) Chatbots and Virtual Assistants such as Siri and Alexa for conversational interaction; and (4) Information Retrieval / Search Engines that rank and retrieve relevant documents for a query. Other notable applications include speech recognition, text summarization, spam detection and automatic question-answering.

9. Define Sentence Segmentation. *[2]*

Sentence segmentation is the process of dividing a block of text into its individual constituent sentences, typically using boundary cues such as full stops, question marks and exclamation marks. It is an important early pre-processing step because many NLP tasks, such as parsing and translation, operate at the level of a single sentence. For example, "NLP is fun. It has many applications." is segmented into two separate sentences. A key challenge is correctly handling ambiguous punctuation, such as periods used in abbreviations (e.g., "Dr.", "U.S."), which should not be treated as sentence boundaries.

10. Explain the importance of text normalization in NLP. *[2]*

Text normalization is essential because raw text collected from real sources (web pages, social media, documents) is inconsistent in case, spelling, punctuation and formatting. By standardizing text — lower-casing, removing extra symbols, expanding abbreviations and correcting variant spellings — normalization ensures that different occurrences of the same underlying word are treated identically by the system. This directly improves the accuracy of tokenization, stemming, parsing and any statistical or machine-learning model trained on the text. Without normalization, the same concept could be represented in many different forms, inflating vocabulary size and reducing model performance.

PART – B (2 × 8 = 16 Marks — Compulsory Questions)

Q6. Apply the complete Natural Language Processing pipeline from text input to meaning representation. Explain the interaction among different processing stages with a suitable example sentence. *[8 Marks]*

Introduction

Natural Language Processing converts raw human language into a structured, machine-interpretable form through a sequence of well-defined stages, commonly called the NLP pipeline. Each stage consumes the output of the previous stage, progressively transforming unstructured text into a formal meaning representation. Applying this pipeline systematically is essential for any real NLP application, from search engines to chatbots, because it decomposes a very hard problem (language understanding) into smaller, more tractable sub-problems.

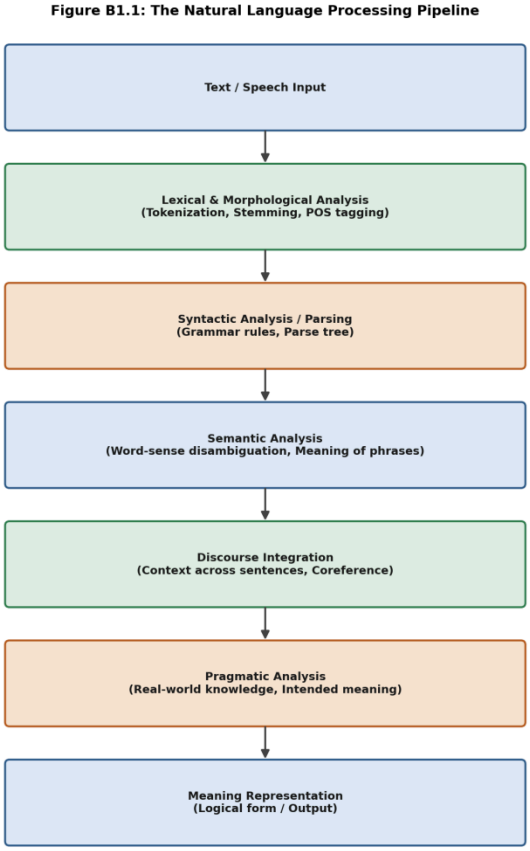

Figure B1.1: The Natural Language Processing pipeline from text input to meaning representation.

Stage-wise Explanation

- **1. Text / Speech Input:** The pipeline begins with raw input — typed text or transcribed speech. This input is unstructured and may contain noise such as typos, special characters, or inconsistent case.

- **2. Lexical and Morphological Analysis:** The text is tokenized into words/sub-words, and each word's internal structure (root, prefix, suffix) is examined. Stemming/lemmatization reduce inflected forms to a base form, and Part-of-Speech (POS) tags are assigned (noun, verb, adjective, etc.).
- **3. Syntactic Analysis (Parsing):** Using grammar rules (e.g., a Context-Free Grammar), the sequence of POS-tagged tokens is organised into a parse tree that shows the grammatical structure — how words group into phrases (NP, VP) and how phrases combine into a sentence (S).
- **4. Semantic Analysis:** The parse tree is mapped to meaning. Word-sense disambiguation selects the correct sense of ambiguous words, and semantic roles (who did what to whom) are identified, producing a literal, context-free meaning representation of the sentence.
- **5. Discourse Integration:** A sentence rarely stands alone. This stage links the meaning of the current sentence to preceding and following sentences, resolving references such as pronouns (coreference resolution) using the broader discourse context.
- **6. Pragmatic Analysis:** The system uses real-world knowledge and the speaker's likely intent to interpret what is actually meant, going beyond the literal meaning — for example recognising that "Can you pass the salt?" is a request, not a question about ability.
- **7. Meaning Representation:** Finally, the fully resolved meaning is expressed in a formal structure (e.g., predicate logic, a semantic frame, or a knowledge-graph triple) that downstream applications can act upon.

Worked Example

Consider the sentence: "The children are playing happily in the park."

- Tokenization: [The, children, are, playing, happily, in, the, park]
- POS tagging: The/DET children/NOUN are/AUX playing/VERB happily/ADV in/PREP the/DET park/NOUN
- Parsing: S → NP("The children") + VP("are playing happily in the park"), where the VP further splits into V("are playing"), ADVP("happily") and PP("in the park").
- Semantic analysis: identifies the agent = "children", action = "play" (ongoing, present tense), manner = "happily", location = "park".
- Discourse integration: if the previous sentence was "I took my kids to the playground," the pipeline links "the children" to "my kids" mentioned earlier.
- Pragmatic analysis: confirms the sentence is a simple statement of fact about an ongoing activity, with no hidden intent to resolve.
- Meaning representation: Play(agent: children, manner: happy, location: park, tense: present-continuous).

Interaction Among the Stages

The stages of the pipeline are not strictly independent — they interact and often need to feed information back and forth. Lexical analysis provides the tokens and POS tags that syntactic analysis depends on to build a correct parse tree; an incorrect POS tag (e.g., tagging “playing” as a noun instead of a verb) would cause the parser to build the wrong tree. Syntactic analysis, in turn, constrains semantic analysis — the grammatical structure tells the semantic analyser which noun phrase is the subject (agent) and which is the object. Semantic analysis often has to consult discourse-level information, for instance to resolve a pronoun before it can assign a semantic role. Finally, pragmatic analysis may override a literal semantic reading based on context, showing that later stages can re-interpret the output of earlier ones. This tightly coupled, sometimes iterative interaction is what allows the pipeline to resolve ambiguity that cannot be settled at any single stage in isolation — for example, syntactic ambiguity (multiple valid parse trees) is often only resolved once semantic and pragmatic knowledge is applied.

The NLP pipeline transforms raw text into formal meaning through a coordinated sequence of lexical, syntactic, semantic, discourse and pragmatic stages, each building upon and sometimes revising the output of the ones before it. This layered, interacting design is what allows modern NLP systems to move from unstructured human language to structured, actionable meaning representations that power real-world applications such as machine translation, question answering and information extraction.

Q6. Illustrate a comprehensive Natural Language Processing (NLP) framework by considering the origins of NLP, and propose suitable solutions to address the major challenges encountered in language processing. ***[8 Marks (Set B)]***

Origins and Evolution of NLP

NLP has evolved over more than seven decades, moving through distinct methodological eras. Understanding this history explains why the modern, comprehensive framework is layered the way it is.

Figure B2.1: Evolution / Origins of Natural Language Processing

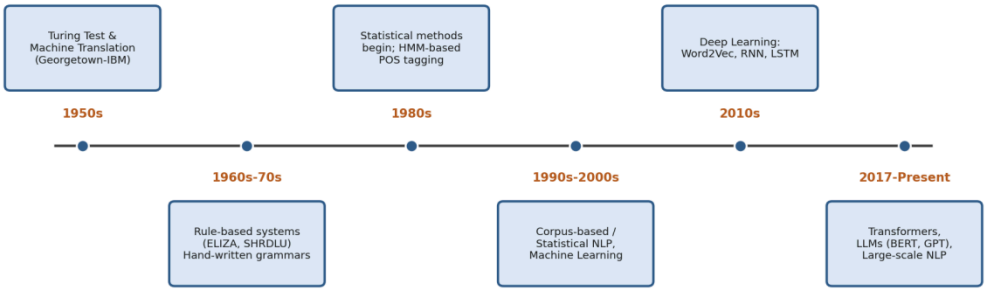

Figure B2.1: Evolution / origins of Natural Language Processing.

- **1950s:** The field's roots trace to Alan Turing's 1950 proposal of the Turing Test, and to early rule-based Machine Translation efforts such as the Georgetown-IBM experiment (1954), which translated Russian sentences to English using hand-coded rules.
- **1960s–70s:** Rule-based and pattern-matching systems dominated, exemplified by ELIZA (a simple chatbot simulating a psychotherapist) and SHRDLU (a system that understood commands about a block world), both relying on hand-written grammars.
- **1980s:** Statistical approaches began to emerge, including Hidden Markov Model (HMM)-based part-of-speech tagging, reducing reliance on purely hand-crafted rules.
- **1990s–2000s:** The availability of large text corpora enabled corpus-based, statistical NLP and classical machine-learning models (Naive Bayes, SVM, CRF) for tasks like classification and tagging.
- **2010s:** Deep learning transformed the field with word embeddings (Word2Vec, GloVe) and sequence models (RNN, LSTM) that learn distributed representations of language automatically.
- **2017–Present:** The Transformer architecture and large pre-trained language models (BERT, GPT and successors) enabled context-aware understanding and generation at a scale not previously possible.

A Comprehensive Layered NLP Framework

Drawing on this evolution, a comprehensive NLP framework can be organised as a layered architecture, where each layer builds on the linguistic processing developed in earlier eras and feeds the one above it, ultimately supporting real-world applications.

Figure B2.2: Comprehensive Layered NLP Framework

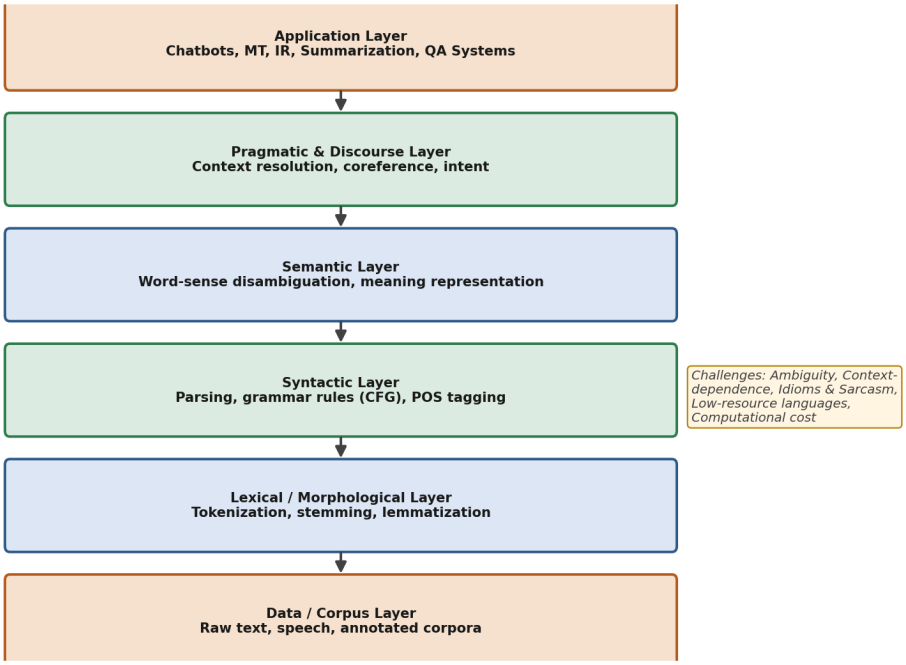

Figure B2.2: A comprehensive layered NLP framework.

- **Data / Corpus Layer:** raw text and speech, annotated corpora, and lexical resources that all higher layers depend on.
- **Lexical / Morphological Layer:** tokenization, stemming and lemmatization that normalise the raw data.
- **Syntactic Layer:** grammar-based parsing and POS tagging that recover sentence structure.
- **Semantic Layer:** word-sense disambiguation and meaning representation.
- **Pragmatic and Discourse Layer:** context resolution, coreference and intent detection.
- **Application Layer:** end-user systems such as machine translation, chatbots, information retrieval and summarization that consume all the layers below.

Major Challenges and Proposed Solutions

Challenge	Description	Proposed Solution
Ambiguity (lexical, syntactic, semantic)	The same word or sentence structure can carry multiple valid meanings.	Use context-aware deep learning models (e.g., Transformer-based contextual embeddings) and probabilistic disambiguation over rigid rule sets.
Context dependence	Meaning often depends on prior discourse or world knowledge not present in the sentence itself.	Employ larger context windows, discourse/coreference resolution modules and knowledge-graph integration.

Challenge	Description	Proposed Solution
Idioms, sarcasm and figurative language	Literal parsing fails to capture intended non-literal meaning.	Train models on annotated sarcasm/idiom datasets and incorporate pragmatic-reasoning components.
Low-resource / multilingual text	Many languages lack large labelled corpora.	Apply transfer learning, cross-lingual embeddings and data augmentation from high-resource languages.
Domain specificity	General models may perform poorly on specialised domains (medical, legal).	Fine-tune / domain-adapt pre-trained models on in-domain corpora.
Computational cost	Large modern models require significant compute and memory.	Use model compression, distillation and efficient architectures for deployment.

A comprehensive NLP framework is best understood as the product of NLP's historical evolution — from hand-written rules, through statistical methods, to today's deep-learning and Transformer-based systems — organised into a layered architecture where each layer resolves a specific aspect of language (lexical, syntactic, semantic, pragmatic). Recognising the major challenges of ambiguity, context-dependence, figurative language, resource scarcity and computational cost, and addressing them with context-aware models, transfer learning and efficient architectures, allows the framework to scale from simple rule-based origins to the robust, general-purpose NLP applications used today.

PART – C (1× 7 = 7 Marks — Both OR Alternatives Answered)

Each Part C question in the original papers offered a choice between two alternatives ("Q.7 OR Q.8"). For completeness, both alternatives are answered below for each paper.

Q7. Analyze the sequence of text pre-processing techniques used in Natural Language Processing. Using the sentence "The children were playing happily in the gardens", illustrate the outputs after tokenization, stop-word removal, stemming, and lemmatization. ***[7 Marks (Set A)]***

Introduction to Text Pre-processing

Text pre-processing is the sequence of transformations applied to raw text before it is fed into any higher-level NLP task such as parsing, classification or information retrieval. The purpose is to remove noise and reduce the vocabulary to a consistent, meaningful set of units, so that models are not confused by superficial variations of the same underlying word. The standard sequence — tokenization, stop-word removal, stemming and lemmatization — is applied in that order because each step depends on the output of the one before it: stop-word removal needs individual tokens to filter, and stemming/lemmatization need the filtered content words to reduce.

Figure C1.1: Text Pre-processing Sequence on the Given Sentence

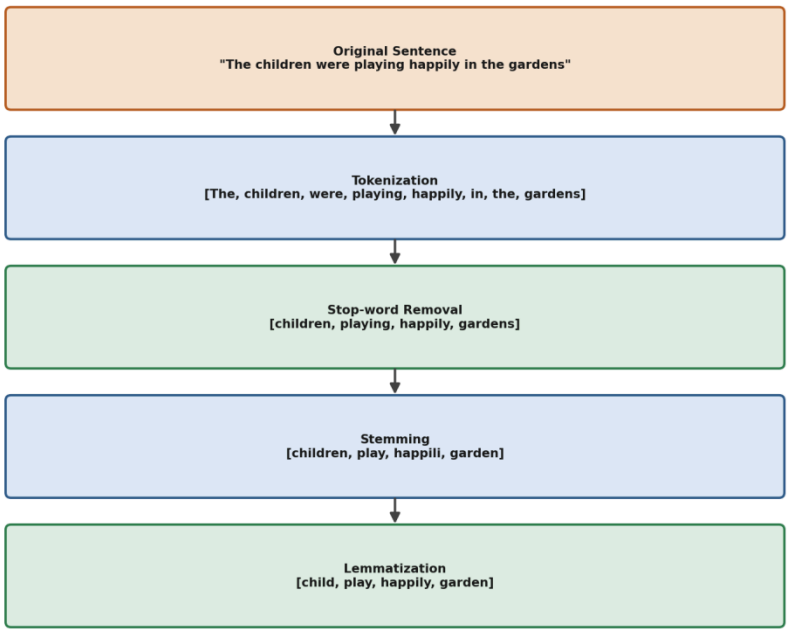

Figure C1.1: Text pre-processing sequence applied to "The children were playing happily in the gardens."

Stage-by-Stage Analysis

- **1. Tokenization:** The raw sentence is split into individual word tokens, breaking on whitespace and punctuation. This is the foundational step that makes every following stage possible, since all later processing operates on discrete tokens rather than the raw character string.

Output: [The, children, were, playing, happily, in, the, gardens]

- **2. Stop-word Removal:** Common function words that carry little independent semantic content — articles, auxiliary verbs, prepositions — are removed using a predefined stop-word list. This reduces the token count and lets subsequent processing focus on the words that actually carry meaning.

Removed: The, were, in, the. Output: [children, playing, happily, gardens]

- **3. Stemming:** Each remaining word is reduced to a crude root form by stripping suffixes using fixed heuristic rules (e.g., a Porter-stemmer style algorithm), without necessarily producing a real dictionary word.

Output: [children → children (stem “children” is irregular and resists simple suffix-stripping), playing → play, happily → happili, gardens → garden]

Result: [children, play, happili, garden] — note “happili” is not a valid English word, illustrating stemming's crude, rule-based nature.

- **4. Lemmatization:** Each word is reduced to its dictionary base form (lemma) using vocabulary and morphological/POS knowledge, so the result is always a valid word.

Output: [children → child, playing → play, happily → happily, gardens → garden]

Result: [child, play, happily, garden] — every output is a proper dictionary word, unlike the stemmed form.

Why This Order Matters

The four techniques must be applied in this specific sequence because each one depends on the structural unit produced by the previous step. Tokenization must come first because stop-word removal, stemming and lemmatization all operate on individual word tokens rather than the raw character string — there is no way to check whether “the” is a stop word until the text has been split into discrete words. Stop-word removal is placed before stemming/lemmatization (rather than after) purely as an efficiency optimisation: it is cheaper to discard function words immediately than to first normalise them and then discard the normalised forms, since stemming/lemmatizing a stop word wastes computation on a word that will be dropped anyway. Finally, stemming and lemmatization are interchangeable in position (only one is normally used in a given pipeline, not both in sequence) because they perform the same conceptual role — reducing a word to a root form — using different techniques.

How the Porter Stemmer Produces “play” and “happili”

A widely used stemming algorithm, the Porter Stemmer, works by applying an ordered series of suffix-stripping rules. For “playing,” the rule that strips the “-ing” suffix from a word whose remaining stem contains a vowel applies directly, yielding “play.” For “happily,” the algorithm applies a rule that

converts a terminal “-ily” to “-ili,” giving “happili” — a form that looks unnatural because the algorithm blindly follows a suffix-substitution rule without checking a real dictionary, which is precisely why stemmed output can be a non-word while lemmatized output cannot.

Additional Worked Example

Applying the same sequence to a second sentence, “The boys are running quickly towards the stadiums,” further illustrates the pipeline:

- Tokenization: [The, boys, are, running, quickly, towards, the, stadiums]
- Stop-word removal: [boys, running, quickly, stadiums]
- Stemming: [boy, run, quickli, stadium]
- Lemmatization: [boy, run, quickly, stadium]

As before, stemming produces the non-standard form “quickli” while lemmatization correctly returns “quickly,” reinforcing that lemmatization's dictionary and POS awareness yields linguistically valid output at the cost of extra processing.

Applications of This Pipeline

- **Search engines:** normalise both queries and documents so that “running” and “run” match the same content.
- **Text classification and sentiment analysis:** reduce vocabulary size so the model generalises better across inflected forms of the same word.
- **Information extraction:** stop-word removal isolates the content words most likely to represent entities and relations.

Stemming vs. Lemmatization — Comparison

Aspect	Stemming	Lemmatization
Method	Rule-based suffix stripping	Dictionary + morphological/POS analysis
Output validity	May produce non-words (e.g., “happili”)	Always a valid dictionary word
Speed	Faster, simpler	Slower, needs linguistic resources
Accuracy	Lower — can over/under-stem	Higher — context and POS aware
Example (this sentence)	children, play, happili, garden	child, play, happily, garden

Conclusion

Applying tokenization, stop-word removal, stemming and lemmatization in sequence to “The children were playing happily in the gardens” shows how each step progressively refines the text: tokenization exposes individual words, stop-word removal isolates the content-bearing words, and stemming/lemmatization normalise those words to a common root — with lemmatization producing

linguistically valid forms at the cost of additional computation compared to the faster but cruder stemming approach.

— OR —

Q8. Illustrate the effectiveness of Bi-gram, Tri-gram, and Four-gram language models in predicting the next word in a sentence. Compare their strengths and limitations, and justify which model is most suitable for different NLP applications. *[7 Marks (Set A — Alternative)]*

N-gram Language Models

An n-gram language model predicts the next word in a sequence based on the previous n–1 words, relying on the Markov assumption that a word's probability depends only on a limited, fixed-size history rather than the entire preceding sentence. The general formula for the probability of a word w given its history is:

$P(w_n \mid w_1 \ldots w_{n-1}) \approx P(w_n \mid w_{n-k} \ldots w_{n-1})$, where k = n–1 is the size of the context window.

Figure C1-OR.1: Context Window Used by Bi-gram, Tri-gram and Four-gram Models

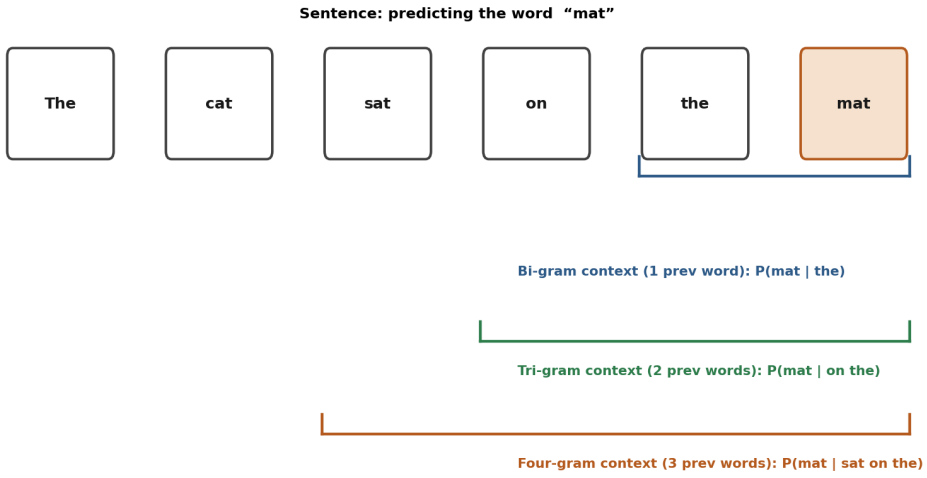

Figure C1-OR.1: Context windows used by bi-gram, tri-gram and four-gram models.

Bi-gram, Tri-gram and Four-gram Models

- **Bi-gram Model (n=2):** Predicts the next word using only the single preceding word, $P(w_n \mid w_{n-1})$. For the sentence "The cat sat on the mat," predicting "mat" uses only "the": P(mat | the).
- **Tri-gram Model (n=3):** Uses the two preceding words, $P(w_n \mid w_{n-2}\, w_{n-1})$. For the same example: P(mat | on the).
- **Four-gram Model (n=4):** Uses the three preceding words, $P(w_n \mid w_{n-3}\, w_{n-2}\, w_{n-1})$. For the same example: P(mat | sat on the).

As the context window widens from bi-gram to four-gram, the prediction becomes more informed by surrounding words, which generally improves accuracy for longer-range dependencies — but at a cost, since wider contexts are seen far less often in any finite training corpus.

Comparison of Strengths and Limitations

Model	Strengths	Limitations
Bi-gram	Low data sparsity, fast to train and query, needs little memory	Captures very limited context; struggles with long-range dependencies and agreement
Tri-gram	Better balance of context and reliability; widely used in practice	More sparsity than bi-gram; still misses dependencies beyond 2 words back
Four-gram	Captures richer local context, useful for more fluent predictions	Severe data sparsity — many four-word sequences never appear in training data, requiring heavy smoothing

Numeric Worked Example

Consider a tiny training corpus of three sentences: “the cat sat on the mat,” “the cat sat on the rug,” and “a dog sat on the mat.” To predict the word following “on the” using a bi-gram model, we only need counts for the pair “the – X”: the word “the” is followed by “mat” twice and “rug” once in the corpus, giving P(mat | the) = 2/3 and P(rug | the) = 1/3 under a simple bi-gram estimate. A tri-gram model instead conditions on “on the” as a unit: “on the” is followed by “mat” once and “rug” once (from the first two sentences) giving P(mat | on the) = 1/2, a more specific but data-sparse estimate since the count is drawn from only two observed instances. Extending to a four-gram, P(mat | sat on the), the exact three-word history “sat on the” appears only twice in this tiny corpus, so the estimate becomes even less statistically reliable — illustrating concretely how widening the context window from bi-gram to four-gram reduces the number of training examples available for each probability estimate.

Handling Data Sparsity: Smoothing

Because higher-order n-grams (tri-gram, four-gram) frequently encounter word sequences that never appeared in the training corpus, a raw probability estimate would incorrectly assign zero probability to any unseen sequence, making the model unable to score a perfectly plausible sentence. Smoothing techniques address this: Laplace (add-one) smoothing adds a small constant count to every possible n-gram; Kneser-Ney smoothing, generally considered more effective, redistributes probability mass based on how many distinct contexts a word appears in. Such smoothing is essential for tri-gram and four-gram models to remain usable despite their inherent sparsity, whereas bi-gram models, having far fewer distinct combinations, are less severely affected.

Justification of Suitability

For applications with limited data or where speed/memory is critical — such as simple predictive text on resource-constrained devices — the bi-gram model is most suitable because of its low sparsity and fast computation, even though its predictions are less context-aware. The tri-gram model offers the best general-purpose trade-off between context and reliability, and is commonly used in classical speech recognition and spelling-correction systems where a reasonable-sized corpus is available. The four-gram model is best suited to applications with very large training corpora and where capturing richer local

context materially improves output quality, such as high-resource statistical machine translation — but it typically requires smoothing techniques (e.g., Kneser-Ney smoothing) to handle the unseen n-grams caused by data sparsity. In modern practice, neural language models have largely superseded fixed n-gram approaches precisely because they avoid this sparsity problem, but understanding n-grams remains foundational to how language modelling works.

Bi-gram, tri-gram and four-gram models illustrate a fundamental trade-off in language modelling between context size and data sparsity: wider context windows improve the linguistic informativeness of predictions but require exponentially more training data to estimate reliably, which is why the right choice of n depends on the size of the available corpus and the accuracy/efficiency requirements of the specific NLP application.

Q7. Analyze the various phases of Natural Language Processing by tracing the sentence "I saw the man with the telescope" through each phase, and explain how ambiguity is introduced, carried forward, or resolved at each stage. *[7 Marks (Set B)]*

Introduction

The sentence "I saw the man with the telescope" is a classic example of structural (syntactic) ambiguity in NLP: it is unclear whether the speaker used a telescope to see the man, or whether the man being seen was carrying a telescope. Tracing this sentence through the standard phases of NLP shows precisely where this ambiguity is introduced, how it persists through later stages, and how it is ultimately resolved.

Figure C2.1: Phases of NLP tracing "I saw the man with the telescope"

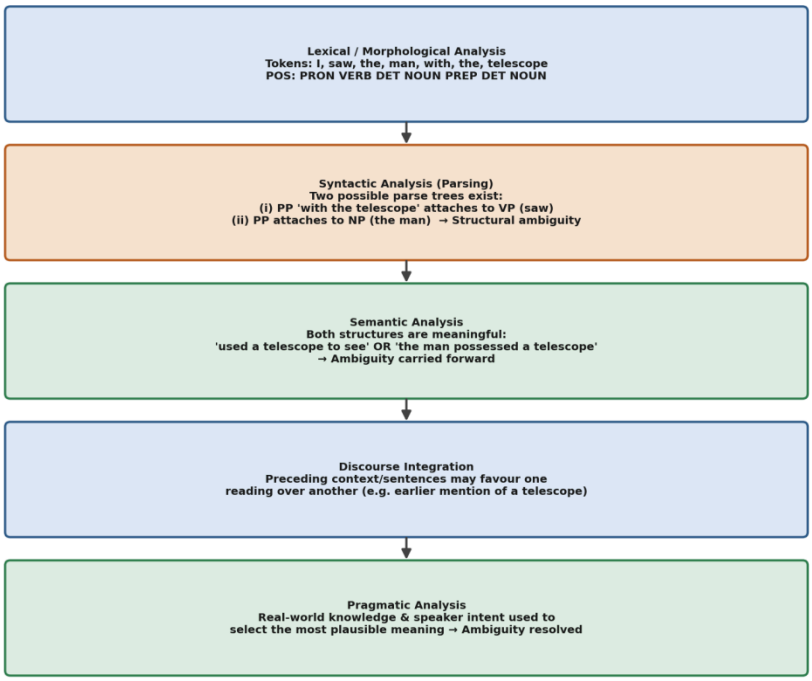

Figure C2.1: Phases of NLP applied to "I saw the man with the telescope."

Phase-wise Trace

- **Lexical / Morphological Analysis:** The sentence is tokenized into [I, saw, the, man, with, the, telescope] and each token is tagged with its part of speech: PRON, VERB, DET, NOUN, PREP, DET, NOUN. At this stage there is no ambiguity yet — every word has a single obvious POS tag in this context.
- **Syntactic Analysis (Parsing):** This is where the ambiguity is introduced. The grammar permits the prepositional phrase "with the telescope" to attach in two different valid places: (i) to the verb phrase, meaning the telescope was the instrument used for seeing, or (ii) to the noun phrase "the man," meaning the man possessed the telescope. Both parse trees are syntactically well-formed, so the parser alone cannot decide between them — this is known as PP-attachment ambiguity.

- **Semantic Analysis:** Both parse trees produce a semantically valid meaning — “I used the telescope as an instrument to see the man” and “I saw the man who had a telescope” are both coherent interpretations. Since neither is semantically impossible, the ambiguity is carried forward rather than resolved here.
- **Discourse Integration:** If this sentence appears within a larger passage, prior context can bias the reading — for example, if an earlier sentence mentioned “I picked up my telescope,” the instrumental reading becomes more likely; if it mentioned “the man was carrying strange equipment,” the possessive reading is favoured. Discourse context therefore narrows, but does not always fully resolve, the ambiguity.
- **Pragmatic Analysis:** Finally, real-world knowledge and the speaker's likely intent are used to settle on the most plausible interpretation given the situation (e.g., in a story about bird-watching, the instrumental reading is far more natural). This is typically the stage at which the ambiguity is fully resolved for practical understanding.

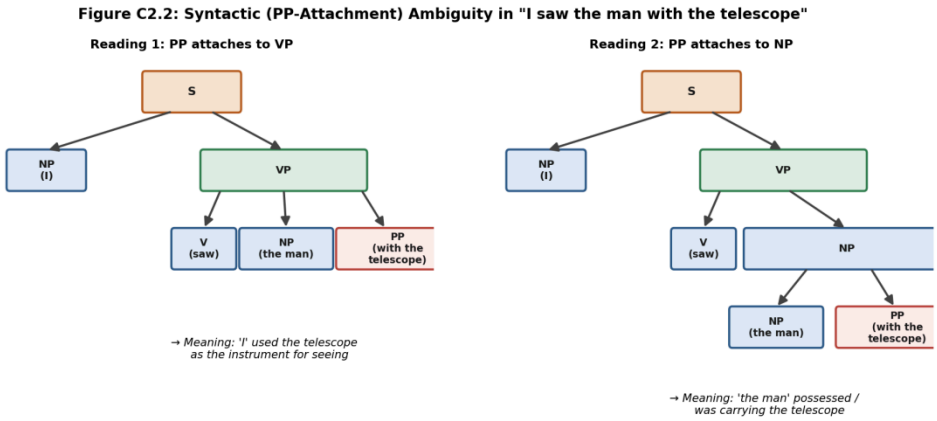

Figure C2.2: The two competing parse trees underlying the PP-attachment ambiguity.

Why the Parser Alone Cannot Resolve This

A syntactic parser built purely on grammar rules (a Context-Free Grammar) has no notion of plausibility or real-world likelihood — it only checks whether a sequence of tokens can be legally derived from the grammar's production rules. Since both attachment structures for “with the telescope” (attaching to the verb phrase, or attaching to the noun phrase “the man”) are equally valid under standard English grammar rules, a purely rule-based parser has no internal mechanism to prefer one over the other, and will either return both trees or arbitrarily pick one. Modern statistical and neural parsers improve on this by learning attachment preferences from large annotated corpora (e.g., they learn that “saw ... with a telescope” is a far more frequent pattern than “man ... with a telescope” in typical text), effectively pulling pragmatic-style, frequency-based knowledge earlier into the syntactic stage — but the fundamental ambiguity illustrated here is still only definitively settled once real-world context is available.

A Second Illustrative Example

The same PP-attachment pattern recurs in many everyday sentences, reinforcing that this is a systematic property of English syntax rather than a one-off quirk of this particular sentence. For instance, “The waiter served the customer with a smile” is ambiguous between “the waiter smiled while serving” (instrumental/manner attachment to the verb) and “the customer who had a smile was served” (attachment to the noun phrase) — exactly mirroring the two-reading structure of the telescope example, and again requiring pragmatic, real-world plausibility (a smiling waiter is far more likely than a smiling customer being specifically identified) to settle on the intended meaning.

Tracing “I saw the man with the telescope” through the phases of NLP shows that ambiguity is not resolved at a single point: it is introduced at the syntactic stage as two competing valid parse trees, carried forward unresolved through semantic analysis because both readings are meaningful, and is only progressively narrowed and finally resolved using discourse context and pragmatic, real-world knowledge — illustrating why later, context-aware stages are indispensable for correct language understanding.

— OR —

Q8. Analyze the different types of ambiguity in Natural Language Processing (NLP). Classify lexical, syntactic, semantic, anaphoric, and pragmatic ambiguities with suitable examples, and explain how each type affects language understanding. *[7 Marks (Set B — Alternative)]*

Introduction

Ambiguity is one of the central challenges in NLP: a single word, phrase or sentence can have more than one valid interpretation, and a system must decide which one the speaker intended. Ambiguity is commonly classified by the linguistic level at which it occurs — lexical, syntactic, semantic, anaphoric and pragmatic — each requiring a different kind of knowledge to resolve.

Figure C2-OR.1: Types of Ambiguity in Natural Language Processing

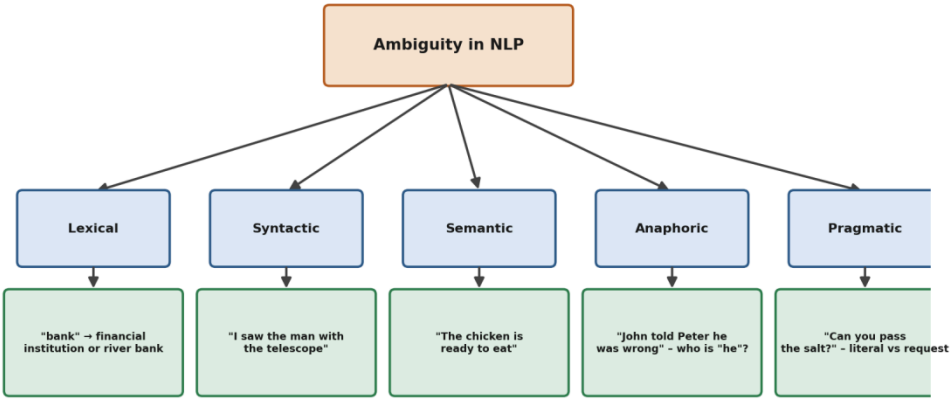

Figure C2-OR.1: The five major types of ambiguity encountered in NLP.

Classification with Examples

- **Lexical Ambiguity:** Arises when a single word has multiple possible meanings (polysemy/homonymy). Example: “bank” can mean a financial institution or the side of a river. Understanding “I deposited money at the bank” requires selecting the correct sense using surrounding words; choosing the wrong sense (word-sense disambiguation failure) leads directly to misunderstanding.
- **Syntactic Ambiguity:** Arises when a sentence can be parsed into more than one valid grammatical structure. Example: “I saw the man with the telescope” can be parsed with the prepositional phrase attaching to either the verb or the noun phrase, producing two different structures and meanings.

- **Semantic Ambiguity:** Arises when a sentence has more than one plausible meaning even after a single parse, often due to word interpretation at the phrase level. Example: “The chicken is ready to eat” can mean the chicken (as food) is ready to be eaten, or the chicken (as an animal) is ready to eat something itself.
- **Anaphoric Ambiguity:** Arises when a pronoun or referring expression could point to more than one earlier-mentioned entity. Example: “John told Peter he was wrong” — the pronoun “he” could refer to either John or Peter, and resolving it correctly requires coreference resolution using discourse context.
- **Pragmatic Ambiguity:** Arises when the literal meaning of an utterance differs from its intended, context-dependent meaning. Example: “Can you pass the salt?” is literally a yes/no question about ability, but is pragmatically almost always intended as a polite request to actually pass the salt.

A Sentence Combining Multiple Ambiguity Types

Real sentences often exhibit more than one type of ambiguity at once. Consider: “The old man's glasses were filled with juice, but he couldn't see them.” Here, “glasses” is lexically ambiguous (spectacles vs. drinking vessels); the pronoun “them” is anaphorically ambiguous (referring to the glasses, or possibly something else mentioned earlier in a longer passage); and the sentence as a whole depends on semantic disambiguation to realise that only the “drinking vessel” sense of “glasses” is consistent with being “filled with juice.” This example shows that resolving one type of ambiguity (lexical) can be a prerequisite for correctly resolving another (anaphoric), since choosing the wrong sense of “glasses” would make the pronoun resolution nonsensical.

Resolution Techniques in More Depth

- **Word-Sense Disambiguation (lexical):** Uses the surrounding words (context window) and knowledge bases such as WordNet to score which sense of an ambiguous word best fits the sentence, often using supervised classifiers or, in modern systems, contextual embeddings from Transformer models.
- **Statistical / Neural Parsing (syntactic):** Rather than returning every grammatically valid parse tree, a probabilistic parser (e.g., a Probabilistic Context-Free Grammar) assigns a likelihood to each tree based on patterns learned from large annotated treebanks, and selects the most probable structure.
- **Selectional Restrictions (semantic):** Checks whether the semantic category of a word fits the role it is being assigned — for example, only an edible entity can plausibly be the object of “to eat,” which helps a system prefer the correct reading of sentences like “The chicken is ready to eat.”
- **Coreference Resolution Algorithms (anaphoric):** Use syntactic cues (e.g., gender, number agreement), recency, and increasingly neural coreference models trained on annotated discourse data, to link a pronoun or referring expression to the correct earlier-mentioned entity.

- **Speech-Act and Intent Modelling (pragmatic):** Uses conversational conventions and world knowledge (e.g., politeness norms) to infer the speaker's communicative goal beyond the literal sentence content, which is central to how virtual assistants correctly interpret indirect requests.

Effect on Language Understanding — Summary

Ambiguity Type	Level	Effect on Understanding	Resolution Approach
Lexical	Word	Wrong word sense chosen changes entire meaning	Word-sense disambiguation using surrounding words
Syntactic	Sentence structure	Multiple parse trees give conflicting meanings	Statistical/probabilistic parsing, semantic filtering
Semantic	Phrase/sentence meaning	Same structure yields more than one coherent meaning	Context and world-knowledge-based disambiguation
Anaphoric	Discourse	Unclear referent leads to incorrect entity linking	Coreference resolution algorithms
Pragmatic	Speaker intent	Literal reading misses the true communicative goal	Pragmatic/intent inference using context and convention

Conclusion

Each type of ambiguity operates at a different linguistic level — from individual words to the speaker's underlying intent — and consequently requires a different resolution strategy, ranging from word-sense disambiguation and statistical parsing to coreference resolution and pragmatic reasoning. A robust NLP system must address all five types together, since real sentences frequently combine more than one form of ambiguity simultaneously, and unresolved ambiguity at any level can propagate and distort the final understanding of the text.